iacc instituto profesional

EVALUACIÓN

Estructuras de Datos.
Semana 4

Amparo Valdivia
01, mayo de 2025
Técnico en Análisis y Programación.

DESARROLLO:

La compañía de seguros "Seguro Protegido" y su módulo de atención al cliente, en la recepción de reclamos y consultas, necesita un sistema de gestión de colas en Python, para reducir los tiempos de espera e implementar las operaciones, insertar y eliminar, analizo y comento que:

- Creo que necesariamente se debería analizar la situación, pensando primeramente en como deberíamos guardar los datos de los clientes, hacer un diccionario con los datos de los clientes, podría ser una muy buena elección, por ejemplo, puedo almacenar la información relevante de cada cliente de forma organizada. ya que no tengo un acceso a una base de datos.
- Con una condición (IF), que sea reclamos o consultas y 2 colas que guarden la información en arrays (listas). La separación en dos colas (reclamos y consultas) es lógica y permitirá gestionar cada tipo de solicitud de manera independiente, lo que podría ser crucial si tienen diferentes prioridades o flujos de atención.
- Más una función que sea para leer, editar, borrar, colocar en histórico, ósea guardar.
- En el primer paso me centraré en la gestión de las colas.
    - Insertar (Encolar): Cuando un cliente contacta a "Seguro Protegido", ya sea para un reclamo o una consulta, se creará un registro (en el diccionario) con la información relevante del cliente y su solicitud. Luego, el ID único de este cliente (o el registro completo) se insertará (encolará) en la cola correspondiente:
    - Si la solicitud es un reclamo, se encolará en la cola de “Reclamos”.

- Si la solicitud es una consulta, se encolará en la cola de “Consultas”.

➢ Eliminar (Desencolar): Cuando un agente de atención al cliente esté disponible para atender una solicitud, se tomará (se eliminará o desencolará) el elemento que esté al frente de la cola correspondiente. Esto asegura el principio FIFO (First-In, First-Out), donde los clientes son atendidos en el orden en que llegaron. El elemento desencolado contendrá la información del cliente (o su ID) que el agente deberá consultar en el diccionario para proceder con la atención.

- El uso de colas asegura que los clientes serán atendidos en el orden en que llegan, evitando que algunos clientes esperen indefinidamente mientras otros son atendidos primero, el primero en llegar será atendido primero, al utilizar colas separadas para reclamos y consultas, se puede implementar una lógica de priorización diferente, igual que en la cola de un consultorio o un banco. Una vez en el banco observé que en el tótem de atención cada persona que llega debe seleccionar el servicio que necesita encolándose su solicitud al respectivo servicio, generando un número de atención.

- El sistema de colas puede proporcionar datos valiosos sobre los tiempos de espera promedio, la longitud de las colas en diferentes momentos, y la cantidad de solicitudes atendidas, lo que permite identificar áreas de mejora en el servicio, y el análisis de los datos.

Código en Python que muestra como puedo gestionar una Cola FIFO.

```
import datetime
cola_reclamos = []
cola_consultas = []

# Encolar un reclamo (simulando la información del cliente, ingresando los datos en un diccionario con clave-valor)
info_cliente_reclamo = {"id": "C001",
                        "nombre": "Ana Pérez",
                        "tipo": "reclamo",
                        "detalle": "Problema con la póliza",
                        "fecha_ingreso" :"2025-05-01 20:14:00",
                        "fecha_egreso" : None}
cola_reclamos.append(info_cliente_reclamo) # Encolamos, append es el push agrega a cola_reclamos, la info_cliente_reclamo.

# Encolar una consulta
info_cliente_consulta = {"id": "C002",
                        "nombre": "Juan Gómez",
                            "tipo": "consulta",
                            "pregunta": "Cobertura del seguro",
                            "fecha_ingreso" :"2025-05-01 20:14:00",
                            "fecha_egreso" : None}#no definido.
cola_consultas.append(info_cliente_consulta) # Encolamos

# Desencolar un reclamo para ser atendido.
if cola_reclamos:
    cliente_a_atender_reclamo = cola_reclamos.pop(0)#saca primer cliente.
    ahora = datetime.datetime.now()#función para saber fecha del instante.
    cliente_a_atender_reclamo["fecha_egreso"]=  ahora.strftime("%Y-%m-%d %H:%M:%S") # Año-Mes-Día Hora:Minuto:Segundo
    print(f"Atendiendo reclamo del cliente con ID: {cliente_a_atender_reclamo['id']}")#imprime el id de quien saco.
else:
    print("No hay reclamos en la cola.")

# Desencolar una consulta para ser atendida
if cola_consultas:
    cliente_a_atender_consulta = cola_consultas.pop(0)
    ahora = datetime.datetime.now()
    cliente_a_atender_consulta["fecha_egreso"]=  ahora.strftime("%Y-%m-%d %H:%M:%S") # Año-Mes-Día Hora:Minuto:Segundo
    print(f"Atendiendo consulta del cliente con ID: {cliente_a_atender_consulta['id']}")
else:
    print("No hay consultas en la cola.")
```

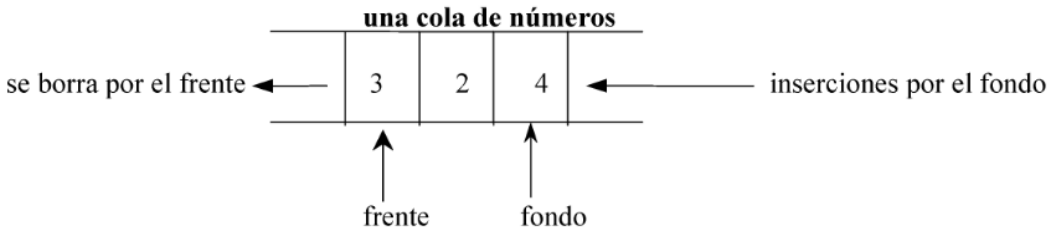
```
historial_cliente=[]#variable que guarda el array de…
def guardar(info_cliente_reclamo,historial):
    historial.append(info_cliente_reclamo)

def sacar_promedio_atencion(historial_cliente):# función sacar promedio atención.
    numero=0
    valor=0
    for cliente in historial_cliente:
        if cliente['fecha_egreso']!=None:
            # Convertir las cadenas a objetos datetime
            fecha_dt1 = datetime.datetime.strptime(cliente['fecha_ingreso'], "%Y-%m-%d %H:%M:%S")
            fecha_dt2 = datetime.datetime.strptime(cliente['fecha_egreso'], "%Y-%m-%d %H:%M:%S")

            # Restar los objetos datetime
            diferencia = fecha_dt2 - fecha_dt1
            valor =valor + diferencia
            numero =numero+1
    if numero==0:
        return None
    else:
        return valor/numero

# si quisiera saber el largo de la cola como ya tengo la hora, se podría con una función sacar el lenght cada 1 hora, por ejemplo.
```

- Como lo menciona Fritelli, V. Guzman, A. y Tymoschuk, J. (2020). Algoritmos y estructuras de datos: (2 ed.). “Las colas son estructuras de datos muy útiles en programas que requieren efectuar lo que se denomina una simulación de situaciones de espera frente a un puesto de servicio.” Ideal para el problema de la atención al cliente.



REFERENCIAS BIBLIOGRÁFICAS

https://rise.articulate.com/share/SDjTUxwIHT7AgRk7L87Ub4a8ff2mXsfj#/lessons/1iWvFu-fr2_476FN3MQNDP3t1JCS3xeZ

Frittelli, V., Guzmán, A. y Tymoschuk, J. (2020). *Algoritmos y Estructuras de Datos.* Jorge Sarmiento Editor - Universitas

https://elibro.net/es/lc/iacc/titulos/175249